Praktikum
Programmierung 3
für Technische Informatik



Konzeption:
Prof. Dr. Oliver Henkel

Durchführung:
Hendrik Plitzner
Timo Schröder

Wintersemester 2025 / 2026

Praktikumsregeln

§1 Ziel des Praktikums ist, Programmierfertigkeiten zu erlernen und durch die kontinuierliche Bearbeitung der Praktikumsaufgaben nachzuweisen. Im Praktikum herrscht deshalb Anwesenheits- und Aktivitätspflicht (praktikumsnahe Aktivität, ansonsten zählt der Termin als ein Fehltermin).

§2 Bis zu 2 Fehltermine werden toleriert und müssen nicht begründet werden. Jeder weitere Fehltermin führt unabhängig vom Fehlgrund zum Nichtbestehen des Praktikums.

§3 Die Bearbeitung der Aufgaben findet einzeln statt. Der Praktikumsbetreuer kann stichprobenweise bearbeitete Aufgaben begutachten. Erfüllt die vorgestellte Lösung die Kriterien einer durch den Praktikumsbetreuer festgelegten Mindestanforderung in nicht ausreichendem Maße, fällt die Begutachtung negativ aus.

§4 Das Praktikum ist sicher bestanden, wenn folgende Bedingungen erfüllt sind:

- Anwesenheit und Aktivität nach §1 und §2 erfüllt
- Sinnvolle und eigenständige Bearbeitung der Aufgaben
- positive Begutachtungen durch den Praktikumsbetreuer

§5 Das Praktikum wird nicht benotet und kann beliebig oft wiederholt werden.

§6 Die Aufgaben sind thematisch gruppiert sollen in dem jeweils angegebenen Zeitraum bearbeitet werden.

§7 Alle notwendigen Daten zur erfolgreichen Bearbeitung der Aufgaben sind in der jeweiligen Aufgabenstellung aufgeführt oder in Form zusätzlicher Dateien den Aufgaben beigefügt. Eine Verwendung von nicht in den Aufgabenstellungen enthaltenen Hilfsmitteln ist nur dann eine zulässige Aufgabenbearbeitung, wenn diese Hilfsmittel detailliert dargelegt, verstanden und in vollem Umfang erklärt werden können.

Kodierregeln

Variablen / Klassenattribute

- Variablen werden nicht für unterschiedliche Zwecke im Programm benutzt. Für jeden einzelnen Zweck ist eine eigene Variable zu definieren.
- Alle verwendeten Namen sind sprechend. Abkürzungen dürfen nur verwendet werden, wenn sie für den Programmierzweck eine eigenständige Bedeutung haben (z.B. physikalische Einheiten, Bezeichner in mathematischen Ausdrücken...)
- Variablen-, Attributs-, Methoden- und Funktionsnamen beginnen mit einem Kleinbuchstaben. Bei aus mehreren Wörtern zusammengesetzten Namen (nicht: zusammengesetzte Wörter wie Startpunkt, Anfangswert) werden ab dem zweiten Wort die ersten Buchstaben jeweils groß geschrieben (Beispiel: `uhrGestartet, zaehlerPunkte`)
- Nur Schleifenvariablen oder Variablen mit einer sehr kurzen Lebensspanne dürfen einfachere Namen haben
- Klassennamen beginnen mit einem Großbuchstaben. Für jede Klasse gibt es (in C++) eine Header- und eine Quellcode-Datei, deren Dateinamen mit dem Klassennamen übereinstimmen
- Konstanten werden groß geschrieben (ggf. durchgehend, mit Unterstrich als Worttrenner)

Inhaltsverzeichnis

1 Entwurfsmuster [2 Wochen] 4

1.1 Zustandsautomat 5

1.2 Beobachter 6

1.3 Dekorator 7

2 generische Programmierung [1 Woche] 9

2.1 Curiously Recurring Template Pattern 10

2.2 nichtnegatives Produkt 12

2.3 Introspektion — bedingte Kompilierung 12

3 STL [1 Woche] 13

3.1 Textstatistik 14

4 Funktoren [1 Woche] 15

4.1 Dekoratoren 16

5 Rekursion [3 Wochen] 17

5.1 Alle Wege 18

5.2 n-th Element 19

5.3 Hanoi mit Memoisation 20

5.4 Binomische Zahlen 21

6 Funktionale Bearbeitung von Datensequenzen[2 Wochen] 23

6.1 Primzahlsieb 24

6.2 Mergesort 25

6.3 Algorithmen 26

7 Test Driven Development[1 Woche] 27

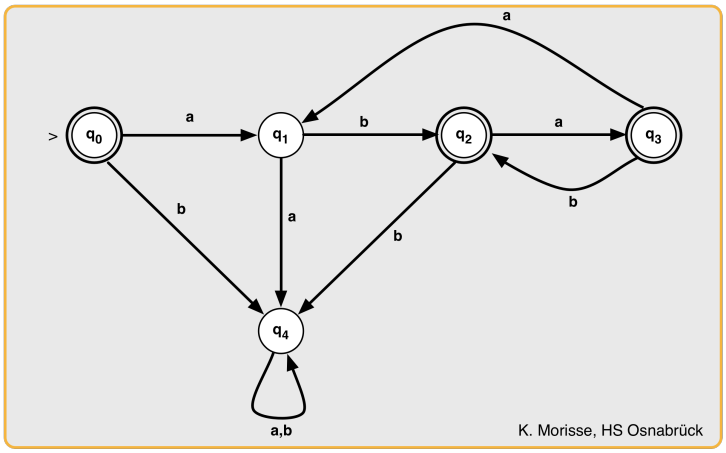
7.1 ...wird im Laufe des Praktikums vorgestellt 28

Kapitel 1

Entwurfsmuster [2 Wochen]

1.1 Zustandsautomat

Simulieren Sie mithilfe des „State“-Entwurfsmusters den folgenden Zustandsautomaten

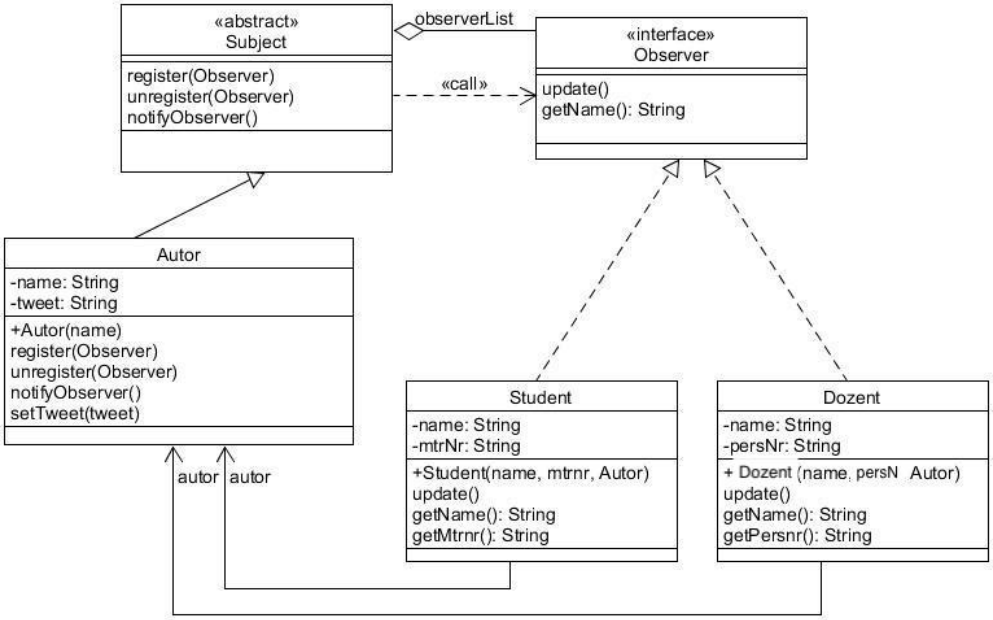


Der mit „>“ markierte Zustand ist der Anfangszustand, doppelt umkreiste Zustände sind anvisierte Endzustände. Alle anderen Zustände sind Zwischenzustände, bzw. Sackgassen.

Die Zustandsübergänge sind die angebenen **char**-Werte a oder b. Testen Sie den Automaten mit verschiedenen „Worten“, z.B. aab, aabb, aba, abb, abba, baa, bab...und bestimmen jeweils, ob ein Endzustand erreicht wird oder nicht. Welche Eigenschaft muss eine Zeichensequenz haben, damit sie vom Anfangs- in einen Endzustand gelangt?

1.2 Beobachter

Entwickeln Sie mithilfe des „Observer“-Musters folgende Anwendung



Konkrete Beobachter (Student/Dozent) können sich bei einem konkreten Subjekt (Autor) an- bzw. abmelden. Ist der Beobachter registriert, empfängt er automatisch Nachrichten, die vom Autor versendet wurden.

1.3 Dekorator

Hintergrund

In Rollenspielen besitzen Helden oft bestimmte Grundwerte (z. B. Lebenspunkte, Mana) und eine Ausrüstung, die diese Werte beeinflussen können. Im Verlauf des Spiels können Helden weitere Ausrüstungsgegenstände wie Schwerter, Rüstungen oder magische Ringe erhalten, welche die Werte weiter verändern. Eine flexible Möglichkeit, solche Ausrüstungen zu modellieren, ist das Dekorator-Pattern. Damit können Objekte "geschmückt" werden, ohne ihre ursprüngliche Klasse verändern zu müssen.

Aufgabenstellung

1. Grundklassen "Held": Implementieren Sie eine abstrakte Klasse `Held`, die folgende Methoden vorgibt:

   - **int** `lebenspunkte`() **const** Gibt die Lebenspunkte zurück
   - **int** `mana`() **const** Gibt die Mana (Zauberkraft) zurück
   - `std::vector<std::string> ausruestung`() **const** Gibt eine Liste aller Ausrüstungsgegenstände zurück

   Erstellen Sie dann zwei konkrete Heldenklassen:

   - Ritter: Startwerte lebenspunkte = 10, mana = 0, Startausrüstung: "100 Gold"
   - Magier: Startwerte lebenspunkte = 5, mana = 1, Startausrüstung: "Rune der Teleportation"

2. Dekoratoren für Ausrüstung

   Implementieren Sie folgende Ausrüstungsgegenstände unter Verwendung des Dekorator-Pattern, welche die Klasse `Held` dekorieren können sollen.

   - Schwert: fügt "Exkalibur" zur Ausrüstung hinzu. Keine Veränderung von Lebensenergie und Mana
   - Rüstung: fügt "Metallrüstung" zur Ausrüstung hinzu und erhöht die Lebenspunkte um +3.
   - MagischerRing: fügt "Magischer Ring" zur Ausrüstung hinzu, erhöht die Lebenspunkte um +2 und Mana um +2.

Anwendungsbeispiel

Die Klassen sollen folgendermaßen verwendet werden können:

```
//Ritter
shared_ptr<Held> ritter = make_shared<Ritter>();
ritter = make_shared<Ruestung>(ritter);
ritter = make_shared<Schwert>(ritter);

//Ausgabe
cout << "--Ritter--" << endl;
cout << "Lebenspunkte: " << ritter->lebenspunkte() << endl;
cout << "Mana: " << ritter->mana() << endl;
cout << "Gegenstände:";
for(const string& s : ritter->ausruestung())
{
    cout << " " << s;
}
cout << endl << endl;

//Magier
shared_ptr<Held> magier
 = make_shared<MagischerRing>(make_shared<Magier>());

//Ausgabe
cout << "--Magier--" << endl;
cout << "Lebenspunkte: " << magier->lebenspunkte() << endl;
cout << "Mana: " << magier->mana() << endl;
cout << "Gegenstände:";
for(const string& s : magier->ausruestung())
{
    cout << " " << s;
}
cout << endl;
```

Und dabei die folgende Ausgabe produzieren:

```
--Ritter--
Lebenspunkte: 13
Mana: 0
Gegenstände: 100 Gold Metallrüstung Exkalibur

--Magier--
Lebenspunkte: 7
Mana: 3
Gegenstände: Rune der Teleportation MagischerRing
```

Kapitel 2

generische Programmierung [1 Woche]

2.1 Curiously Recurring Template Pattern

1. Aufwärmübung: `InstanceCounter<Derived>`
   Ziel: Verstehen, dass eine CRTP-Basisklasse für jede abgeleitete Klasse eine eigene, unabhängige Instanziierung erzeugt (im Gegensatz zu gewöhnlicher Vererbung von einer gemeinsamen Basisklasse).

   Implementieren Sie eine Klassenschablone `InstanceCounter<Derived>`, die mitzählt, wie viele lebende Objekte des jeweiligen `Derived`-Typs aktuell existieren. Erstellen Sie dann zwei (leere) Klassen
   class `Widget` : **public** `InstanceCounter<Widget>` ;
   class `Gadget` : **public** `InstanceCounter<Gadget>` ;

   Folgende Testfälle sollen erfüllt sein:

```
{
 Widget w1, w2;
 Gadget g1;
 assert(Widget::aktuelle_anzahl() == 2);
 assert(Gadget::aktuelle_anzahl() == 1);
}
assert(Widget::aktuelle_anzahl() == 0);
assert(Gadget::aktuelle_anzahl() == 0);
```

   Leitfragen zur Reflexion:
   Warum haben `Widget` und `Gadget` unabhängige Zähler, obwohl beide von derselben "Klasse" `InstanceCounter` erben?
   Was würde passieren, wenn `InstanceCounter` nicht schabloniert wäre, sondern eine gewöhnliche gemeinsame Basisklasse `InstanceCounterBase` mit einem einzigen `static int count`? (ggf. ausprobieren)

2. Anreicherung von Methoden durch eine Basisklasse
   Implementieren Sie die CRTP-Basisklasse `Comparable<Derived>`. Voraussetzung ist, dass `Derived` die Methoden
   bool `equals`(**const** `Derived`& `other`) **const**;
   bool `less_than`(**const** `Derived`& `other`) **const**;
   bereitstellen. `Comparable<Derived>` soll daraus automatisch die Operatoren `==, !=, <, <=, >, >=` bereitstellen — ohne dass `Derived` diese selbst implementieren muss.

   Folgendes Beispiel mit Tests muss laufen:

```
class Bruch : public Comparable<Bruch> {
 int zaehler, nenner;
```

```
public:
 Bruch(int z, int n) : zaehler(z), nenner(n) {}

 bool equals(const Bruch& b) const {
  return zaehler * b.nenner == b.zaehler * nenner;
 }
 bool less_than(const Bruch& b) const {
  return zaehler * b.nenner < b.zaehler * nenner;
 }
};

// Testfälle:
Bruch a(1, 2), b(2, 4), c(3, 4);
assert(a == b);
assert(a != c);
assert(a < c);
assert(c > a);
assert(a <= b);
assert(c >= a);
```

2.2 nichtnegatives Produkt

Verwenden Sie Konzepte (`concept`), für die Prüfung der Anforderung, das T ein nichtnegativer ganzer Zahltyp ist oder ein Gleitpunkt-Typ ist.

Schreiben Sie eine Funktionsschablone, die für zwei solcher T-Werte das Produkt berechnet und zurückgibt, sowie eine Überladung der Schablone für zwei Parameter vom Typ `list<T>`, die das Produkt der beiden ersten, zweiten, usw. Listenelemente bildet, sofern die Listen die gleiche Anzahl von Elementen haben und das Ergebnis als Liste zurückliefert.

Die Erfüllung der Anforderungen an den Schablonentyp soll bereits durch den Compiler überprüft werden.

2.3 Introspektion — bedingte Kompilierung

Implementieren Sie mithilfe von Konzepten (`concept`) eine bedingte Kompilierung für ein Programm, dass abhängig davon, ob ein Container die Methode `push` oder `push_back` zur Verfügung stellt, die passende Methode aufruft.

Die Container sollen jeweils Elemente vom Typ int haben.

Kapitel 3

STL [1 Woche]

3.1 Textstatistik

Schreiben Sie ein Programm `textstatistik.cpp`, das mithilfe geeigneter assoziativer Container folgendes leistet: Eine einzulesende Textdatei (Testfälle sind die Datei `glocke` und `buergschaft`) soll analysiert werden.

a) Für jeden vorkommenden Buchstaben (in Kleinbuchstaben konvertiert) soll die Häufigkeit bestimmt werden.

b) Alle Worte sollen gesammelt werden und auf dem Bildschirm eine Tabelle erstellt werden, die für die Werte n=2 bis n=20 die Anzahl der Worte mit dieser Wortlänge angibt. Auf eine Benutzereingabe von n hin sollen alle Worte mit n Buchstaben ausgegeben werden.

Hinweis: Eine geschickte Containerwahl vereinfacht den Algorithmus erheblich

- Zwischen Groß- und Kleinschreibung von Worten wird nicht unterschieden
- Worte enthalten mindestens zwei Buchstaben und weder Satz- noch Sonderzeichen
- Doppelt vorkommende Worte sollen natürlich nur einmal (gespeichert und) ausgegeben werden
- Versuchen Sie, mit so wenig Schleifen/Iterationen wie möglich auszukommen

Kapitel 4

Funktoren [1 Woche]

4.1 Dekoratoren

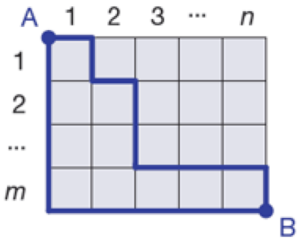
Implementieren Sie eine funktionale Variante des Dekorators für die Zusammenstellung von (Haupt-)Gerichten und Beilagen angelehnt an das OOP-Beispiel aus der Vorlesung. Sie können annehmen, dass jedes Gericht eine Funktion ist, die ein Paar, bestehend aus dem Namen des Gerichts und dem Preis, zurückliefert.

Kapitel 5

Rekursion [3 Wochen]

5.1 Alle Wege

Entwickeln Sie eine rekursive Funktion, die die Anzahl aller (kürzesten) Wege entlang der Gitterlinien von der oberen linken Ecke A zur unteren rechten Ecke B bestimmt.



Beantworten Sie im Zuge der Problemlösung die Fragen:

- Welche Fälle sind Basisfälle?
- Wie sieht das rekursive Bildungsgesetz aus?

5.2 n-th Element

Bestimmen Sie aus einer Liste von Integer-Werten (Typ `list<int>`) und Eingabe einer Zahl n das n.-kleinste Listenelement, d.h. denjenigen Wert, der bei einer Sortierung der Liste an der n-ten Stelle stünde.

Es darf aber nicht sortiert werden. Stattdessen sollen Sie einen rekursiven Algorithmus nach folgender Idee implementieren: Die Liste wird rekursiv aufgeteilt in eine Liste mit Elementen, die kleiner als das erste Listenelement sind und die Liste der Elemente, die größer als das erste Listenelement (im folgenden: das Pivot-Element) sind.

Besitzt die Menge mit den kleineren Elementen genau $n - 1$ Elemente, so ist das Pivot-Element das gesuchte. Besitzt die Menge mit den kleineren Elementen mehr als $n - 1$ Elemente, so ist das gesuchte Element in dieser Menge zu finden (nämlich auch dort das n-t Kleinste), d.h. es kann rekursiv diese (kleinere) Liste nach demselben Schema aufgeteilt werden. Besitzt die Liste mit den kleineren Elementen jedoch weniger als $n-1$ Elemente, so muss die Menge mit den größeren Elementen rekursiv aufgeteilt werden.

5.3 Hanoi mit Memoisation

Erstellen Sie für die rekursive Fassung der Türme von Hanoi aus der Vorlesung eine Variante mit Memoisation.

5.4 Binomische Zahlen

Schreiben Sie ein Programm, dass nach Eingabe einer ganzen Zahl n die Binomialkoeffizienten $\binom{n}{k}$ für $k = 0$ bis $k = n$ ausgibt. Die Binomialkoeffizienten sind für natürliche Zahlen n und $k \leq n$ rekursiv definiert durch:

$$\binom{n}{0} = 1\,, \quad \binom{n}{1} = n\,, \quad \binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

und Null in allen anderen Fällen.

- Schreiben Sie dafür eine rekursive Funktion `binomi_rekursiv(n,k)`, die die Berechnung rekursiv durchführt.
- Schreiben als eine Version `binomi_Stapel(n,k)`, die die Rekursion iterativ mithilfe eines Stapelspeichers imitiert
- Schreiben Sie eine performantere iterative Variante `binomi_iterativ(n,k)`, die sukzessive die Werte in den Zeilen des Pascalschen Dreiecks berechnet, bis die Stelle (n,k) erreicht ist. Benutzen Sie zur Vereinfachung die Symmetrie $\binom{n}{k} = \binom{n}{n-k}$, um für die Berechnung stets $k \leq n - k$ annehmen zu können.

  Folgende Überlegung ist hilfreich:

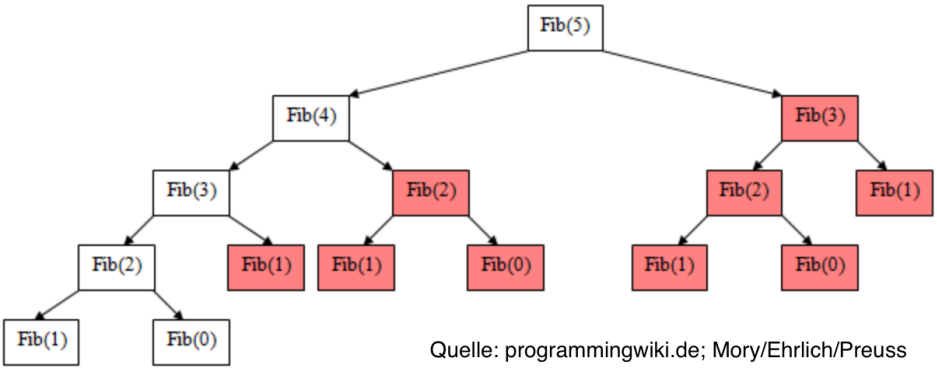
	k			
	1	1		
	1	2	1	
	1	3	3	1
n	1	4	6	4
	1	5	10	10
	1	6	15	20
	1	7	21	35
	1	8	28	56

  Der Binomialkoeffizient $\binom{n}{k}$ ergibt sich zeilenweise aus der vorhergehenden Zeile. Es müssen dabei nur für die ersten $\min(k+1, n-k+1)$ Werte ausgerechnet werden. Es genügt also, einen Zwischenspeicher für jeweils $k+1$ Werte anzulegen und dann zeilenweise aus diesen Werten die jeweiligen Werte für die nachfolgende Zeile zu berechnen. Der oben abgebildeten Skizze können Sie beispielhaft die iterative Berechnung von $\binom{8}{5}$ entnehmen.

Ihr Programm soll sicher sein, d.h. für jede numerische Eingabe korrekt arbeiten und ohne globale Variablen auskommen. Die entrekursivierte Fassung soll nach der oben beschriebenen Anleitung angefertigt werden.

Hinweis: Der größte Binomialkoeffizient, der noch in eine Variable vom Typ integer passt, ist $\binom{33}{17} = 1.166.803.110$

Bemerkung:
Die Implementation nach Art `binomi_iterativ(n,k)` nennt man dynamische Programmierung. Diese Lösungsstrategie besteht darin, zunächst Teillösungen (ggf. auch rekursiv) zu bestimmen und zwischenzuspeichern. Die Gesamtlösung wird dann aus den Teilen zusammengesetzt. Bei den in der Vorlesung vorgestellten Formen der iterativen Berechnungsmethoden der Fibonaccizahlen geschieht dies durch Wiederholung der Sequenz `g=f1+f2; f1=f2; f2=g;`, wobei die aktuellen Zwischenergebnisse nach jedem Schleifendurchlauf gerade den Werten von `f1` und `f2` bestehen und die Berechnung längs des linken (ungefärbten) Teilbaums im Rekursionsbaum stattfindet.



Quelle: programmingwiki.de; Mory/Ehrlich/Preuss

Kapitel 6

Funktionale Bearbeitung von Datensequenzen[2 Wochen]

6.1 Primzahlsieb

Implementieren Sie das Primzahlsieb des Eratosthenes: Aus der Liste der Zahlen von 2 bis N werden sukzessive alle Vielfachen gestrichen, bis nur noch Primzahlen übrig bleiben.

Die zu siebende Liste der Zahlen von 2 bis `N` soll im aufrufenden Prograll als `range` erzeugt und an die Funktion übergeben werden. Die Sieboperationen sollen durch Filteroperationen einer `view` erfolgen.

6.2 Mergesort

Implementieren Sie analog zum Vorgehen beim Quicksort in der Vorlesung den Mergesort-Algorithmus für eine zu sortierende Liste von integer-Werten.

Die Funktion soll die zu sortierende Liste als `range` übergeben bekommen, sowie ein Sortierkriterium (kleiner, größer, Betragskleiner). Das Sortierkriterium soll im aufrufenden Programm jeweils als Funktionszeiger und als Lambdafunktion vorliegen und mittels `std::function` an den Mergesort-Algorithmus übergeben werden.

6.3 Algorithmen

Verwenden Sie die Algorithmen/Projektionen der Ranges-Bibliothek:

- In einer Liste von Läufern (Name, Zeit für 100m) soll der Läufer mit der n-kleinsten Laufzeit ermittelt werden. n wird vom Benutzer eingegeben.
- Schreiben Sie den Quicksort-Algorithmus aus der Vorlesung so um, dass der `partition`−Algorithmus verwendet wird.

Kapitel 7

Test Driven Development[1 Woche]

7.1 ...wird im Laufe des Praktikums vorgestellt